# Proiectarea Algoritmilor

II CA: Traian Rebedea traian.rebedea@upb.ro

II CB: Radu Cristian Alexandru Iacob radu.iacob@upb.ro

II CC: Costin Chiru costin.chiru@upb.ro

II CD: Andrei Mogos andrei.mogos@upb.ro



# Obiectivele cursului (I)

- Cunoașterea unui set important de algoritmi și metode de rezolvare a problemelor de algoritmică. → curs
- Dezvoltarea abilităților de adaptare a unui algoritm la o problemă din viața reală.→ laborator/teme/proiect/test practic
- Dezvoltarea abilităților de lucru în echipă.→ proiect



# Obiectivele cursului (II)

- Utilizarea teoriei predate la curs pentru proiectarea algoritmilor de rezolvare pentru probleme tipice întâlnite în practica dezvoltării sistemelor de programe.
- Discutarea relaţiei dintre caracteristicile problemelor, modul de rezolvare şi calitatea soluţiilor.
- Compararea variantelor unor algoritmi pentru rezolvarea problemelor dificile.
- Demonstrarea corectitudinii folosirii algoritmilor aleși și a eficienței acestora.



# De ce să învăț PA?

- Exemple de utilizări ale PA-ului in diferite meserii:
  - web developer – web social, teoria grafurilor, data mining, clustering;
  - game dev – căutări, grafuri, inteligență artificială;
  - project manager – fluxuri, grafuri de activități;
  - dezvoltator de sisteme de operare – structuri de date avansate, scheme de algoritmi;
  - programator – tot ce ține de algoritmi, în special complexitate și eficiență;
  - tester – tot ce ține de algoritmi, în special complexitate, eficiență și debugging;



# Planul cursului (I)

- Scheme de algoritmi
  - Caracteristici ale problemelor şi tehnici asociate de rezolvare: divide & impera (Merge Sort, puterea unui număr), rezolvare lacomă (arbori Hufmann, problema rucsacului continuă), programare dinamică (înmulțirea matricelor, AOC, problema rucsacului discretă).
- Algoritmi pentru grafuri
  - Parcurgeri, drumuri de cost minim, arbori minimi de acoperire, sortare topologică, componente tare conexe, puncte de articulație, punți, fluxuri.



# Planul cursului (II)

- Rezolvarea problemelor prin căutare euristică
  - Rezolvarea problemelor prin căutarea euristică A*. Completitudine și optimalitate, caracteristici ale euristicilor.
- Backtracking – Optimizări. Propagarea restricțiilor.
- Algoritmi pentru jocuri – Minimax și $\alpha$-$\beta$.
- Algoritmi aleatorii
  - Las Vegas și Monte Carlo. Aproximare probabilistică.



# Evaluarea

- Citiți documentul “Regulament Proiectarea Algoritmilor 2023” de pe site! Acesta va fi disponibil la adresa: http://ocw.cs.pub.ro/courses/pa/regulament-general
- Examen 4 p (minim 2 p pentru promovare, minim 3 p pentru a putea lua nota 10)
- Laborator 6 p ☺ (minim 3 p pentru promovare)
  - 2.5 p teme (2 teme punctate egal) (+ 0.5 p bonus distribuit egal)
  - 1.5 p test practic
  - 1 p proiect (+1 p bonus)
  - 1 p laborator - activitate efectivă în laborator
- Bonus maxim 0,5 – 1,5 p (în funcție de realizări): Rezultate bune la concursuri de programare relevante / activitate științifică remarcabilă
- Obs. 1 - Nu se copiază în facultate!
- Obs. 2 - Prima temă copiată se punctează cu minus valoarea maximă a temei. La a doua temă copiată, se repetă materia!



# Proiect PA

- Tema proiectului: TBA

- Documentul aferent proiectului va fi disponibil la adresa:
- http://ocw.cs.pub.ro/courses/pa/regulament-proiect

- Obiective:
  - Rezolvarea unei probleme interesante din viaţa reală;
  - Găsirea și aplicarea unor euristici pentru această problemă;
  - Dezvoltarea abilităților de lucru în echipă;
  - Dezvoltarea spiritului competitiv.

- Mai multe informații în “Regulament Proiect PA 2023” (disponibil probabil la finalul săptămânii)



# Feedback 2022 - Note

- Cadrul didactic stăpânește bine domeniul de studiu?
  - Curs 4.44 / 5
  - Laborator 4.69 / 5
- Metoda de expunere a fost potrivită la curs? 3.64 / 5
- Cadrul didactic a sprijinit activitatea individuală a studenților la laborator? 4.54 / 5
- Cursul / Aplicațiile au stimulat discuțiile și cadrul didactic a răspuns clar întrebărilor studenților?
  - Curs 4 / 5
  - Laborator 4.6 / 5
- Comportamentul cadrului didactic față de studenți a fost adecvat?
  - Curs 4.84 / 5
  - Laboratoare 4.76 / 5
- Materialele puse la dispoziție sunt suficiente pentru înțelegerea?
  - Curs 4.04 / 5
  - Laboratoare 4.36 / 5
- Numărul și dificultatea temelor au fost adecvate? 2.84 / 5
- Temele/proiectele/activitățile practice au ajutat la înțelegerea materiei? 3.6 / 5

# Feedback 2022 - Aspecte pozitive

- Echipa este foarte bine organizată.
- Curs:
  - S-a răspuns clar la toate întrebările. Nu au existat ezitări. Domnul profesor stăpânește materia. ("Mi-a plăcut că am înțeles algoritmii de la grafuri care anul trecut la SD mi se păreau imposibili și ciudați.")
- Laboratoare:
  - Sunt foarte bine făcute, mai ales checkerele.
  - Se poate programa atât în C++ cât și în Java.
  - A fost mai mult sub formă de seminar și mi s-a părut foarte util să mi se explice idei și să se parcurgă algoritmii pas cu pas pe exemple.
  - Punctarea: nu se mai pun note pe câte probleme rezolvi la laborator și poți să te concentrezi cu adevărat pe înțelegerea conceptelor, și nu te simți stresat și nevoit să alergi după o notă.
- Deadline-urile au fost super bine puse, pentru teme și pentru testul practic.
- Proiectul (Halite) - lucru pe echipe + a fost fun



# Feedback 2022 - Aspecte negative

- Încărcare: O materie ce necesită destul de mult timp → destul de greu având și PP / PCom. ("Nivelul de încărcare pe care această materie îl pune pe studenți e unul pe care eu nu l-am mai întâlnit până acum în facultate.")
- Testul practic are dificultate mare: Problemele medii erau prea grele.
- Proiectul a fost o glumă proastă. Se poate observa și din câți oameni l-au rezolvat.
- Curs:
  - Demonstrații pe care nimeni nu le înțelege. Nu ne ajută la nimic. Vrem chestii practice: unde se folosesc în industrie, ce optimizări există. Nu interesează pe nimeni demonstrațiile lor. Sunt inutile. Vrem chestii palpabile.
  - Aspectele teoretice de la curs sunt noi și destul de dificil de înțeles.
  - Să se insiste mai mult pe DP. Simt că nu a înțeles aproape nimeni cum se folosește.
- Sugestii:
  - S-ar putea crea o platformă cu probleme de algoritmică (tip hackerrank) unde să existe un pool cu mai multe probleme de algoritmică și în funcție de câte rezolvi, la finalul semestrului, să primești un anumit punctaj (eventual bonus).
  - Poate în loc de proiect se mai face o temă.

# Bibliografie

- Introducere in Analiza Algoritmilor de *Cristian Giumale* – Ed. Polirom 2004
- Introducere in Algoritmi de *Thomas H. Cormen, Charles E. Leiserson, Ronald R. Rivest, Clifford Stein* – Ed. Agora (preferabil ediția a 3a) 2009
  - Varianta în engleză de la MIT Press
- Vedeţi şi recomandările din Regulament!
- Mențiune importantă – **slide-urile reprezintă doar un suport pentru prezentare!**

# Proiectarea Algoritmilor

Curs 1 – Scheme de algoritmi – Divide et impera



# Curs 1 – Cuprins

- Scheme de algoritmi
- Divide et impera
- Exemplificare folosind Merge sort
- Alte exemple de algoritmi divide et impera



# Curs 1 – Bibliografie

- Giumale – Introducere in Analiza Algoritmilor cap 4.4
- Cormen – Introducere în Algoritmi cap. Algoritmi Greedy (17)
- http://www.cs.ucsb.edu/~suri/cs235/ClosestPair.pdf
- http://www.cs.rit.edu/~ib/Classes/CS515_Spring12-13/Slides/022-SelectMasterThm.pdf



# Scheme de algoritmi

- Prin scheme de algoritmi înțelegem tipare comune pe care le putem aplica în rezolvarea unor probleme similare.
- O gamă largă de probleme se pot rezolva folosind un număr relativ mic de scheme.
- => Cunoașterea schemelor determină o rezolvare mai rapidă și mai eficientă a problemelor.

# Divide et Impera (I)

- Ideea (divide si cucerește) este atribuită lui Filip al II-lea, regele Macedoniei (382-336 i.e.n.), tatăl lui Alexandru cel Mare și se referă la politica acestuia față de statele grecești.

- În CS – Divide et impera se referă la o clasă de algoritmi care au ca principale caracteristici faptul că împart problema în subprobleme similare cu problema inițială dar mai mici ca dimensiune, rezolvă problemele recursiv și apoi combină soluțiile pentru a crea o soluție pentru problema originală.

# Divide et Impera (II)

- Schema Divide et impera constă în 3 pași la fiecare nivel al recurenței:
  - Divide problema dată într-un număr de subprobleme;
  - Impera (cucerește) – subproblemele sunt rezolvate recursiv. Dacă subproblemele sunt suficient de mici ca date de intrare se rezolvă direct (ieșirea din recurență);
  - Combină – soluțiile subproblemelor sunt combinate pentru a obține soluția problemei inițiale.



# Divide et Impera – Avantaje și Dezavantaje

- Avantaje:
  - Produce algoritmi eficienți.
  - Descompunerea problemei în subprobleme facilitează paralelizarea algoritmului în vederea execuției sale pe mai multe procesoare.
- Dezavantaje:
  - Se adaugă un overhead datorat recursivității (reținerea pe stivă a apelurilor funcțiilor).



# Merge Sort (I)

- Algoritmul Merge Sort este un exemplu clasic de rezolvare cu D&I.
- Divide: Divide cele n elemente ce trebuie sortate în 2 secvențe de lungime n/2.
- Impera: Sortează secvențele recursiv folosind *merge sort.*
- Combină: Secvențele sortate sunt asamblate pentru a obține vectorul sortat.
- Recurența se oprește când secvența ce trebuie sortată are lungimea 1 (un vector cu un singur element este întotdeauna sortat ☺) .
- Operația cheie este: asamblarea soluțiilor parțiale.

# Merge Sort (II)

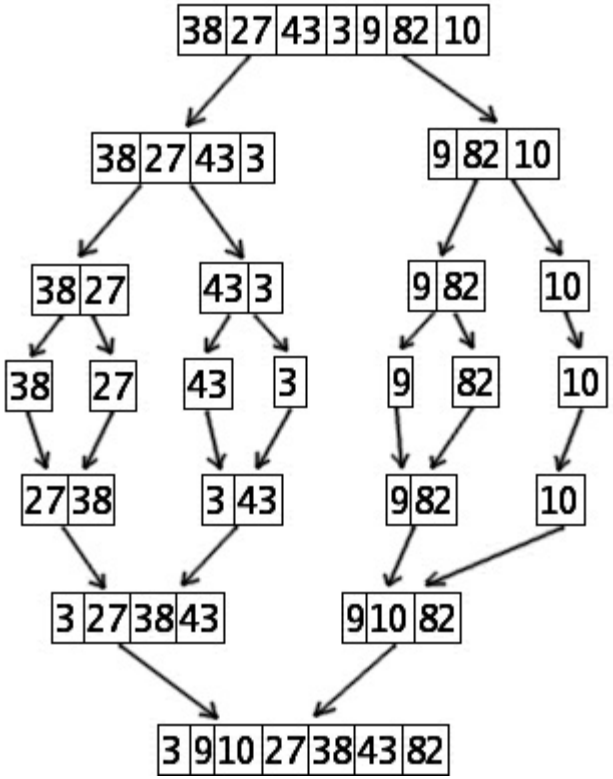
- Algoritm [Cormen]
  - MERGE-SORT($A$, $p$, $r$)
  - 1 **Dacă** $p < r$
  - 2 **Atunci** $q \leftarrow [(p + r)/2]$ // divide
  - 3 MERGE-SORT($A$, $p$, $q$) //impera
  - 4 MERGE-SORT($A$, $q + 1$, $r$)
  - 5 MERGE($A$, $p$, $q$, $r$) // combină
    // (interclasare)

# Merge Sort (III) – Algoritmul de interclasare

- Algoritm [Cormen]
  - MERGE($A$, $p$, $q$, $r$) // p si r sunt capetele intervalului, q este “mijlocul”
  - 1 $n_1 \leftarrow q - p + 1$ // numărul de elemente din partea stânga
  - 2 $n_2 \leftarrow r - q$ // numărul de elemente din partea dreapta
  - 3 **creează** vectorii $S[1 \rightarrow n_1 + 1]$ și $D[1 \rightarrow n_2 + 1]$
  - 4 **Pentru** $i$ **de la** 1 **la** $n_1$
  - 5 $S[i] \leftarrow A[p + i - 1]$ // se copiază partea stânga în S
  - 6 **Pentru** $j$ **de la** 1 **la** $n_2$
  - 7 $D[j] \leftarrow A[q + j]$ // si partea dreapta în D
  - 8 $S[n_1 + 1] \leftarrow \infty$
  - 9 $D[n_2 + 1] \leftarrow \infty$
  - 10 $i \leftarrow 1$
  - 11 $j \leftarrow 1$
  - 12 **Pentru** $k$ **de la** $p$ **la** $r$ // se copiază înapoi în vectorul de
  - 13 **Dacă** $S[i] \leq D[j]$ // sortat elementul mai mic din cei
  - 14 **Atunci** $A[k] \leftarrow S[i]$ // doi vectori sortați deja
  - 15 $i \leftarrow i + 1$
  - 16 **Altfel** $A[k] \leftarrow D[j]$
  - 17 $j \leftarrow j + 1$

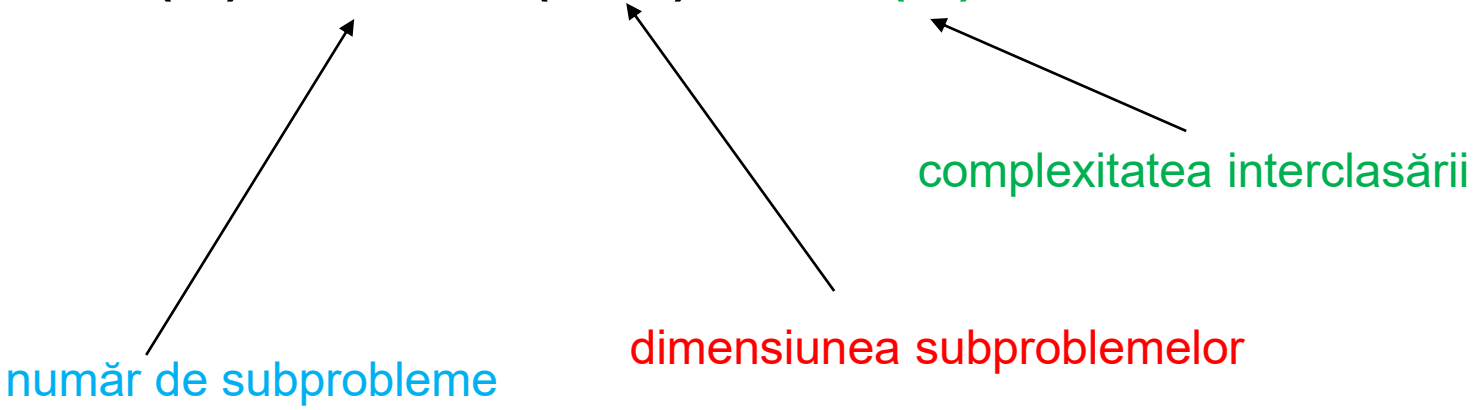
# Exemplu funcționare Merge Sort

- Exemplu funcționare [Wikipedia]:



# Merge Sort - Complexitate

- $T(n) = 2 * T(n/2) + \Theta(n)$



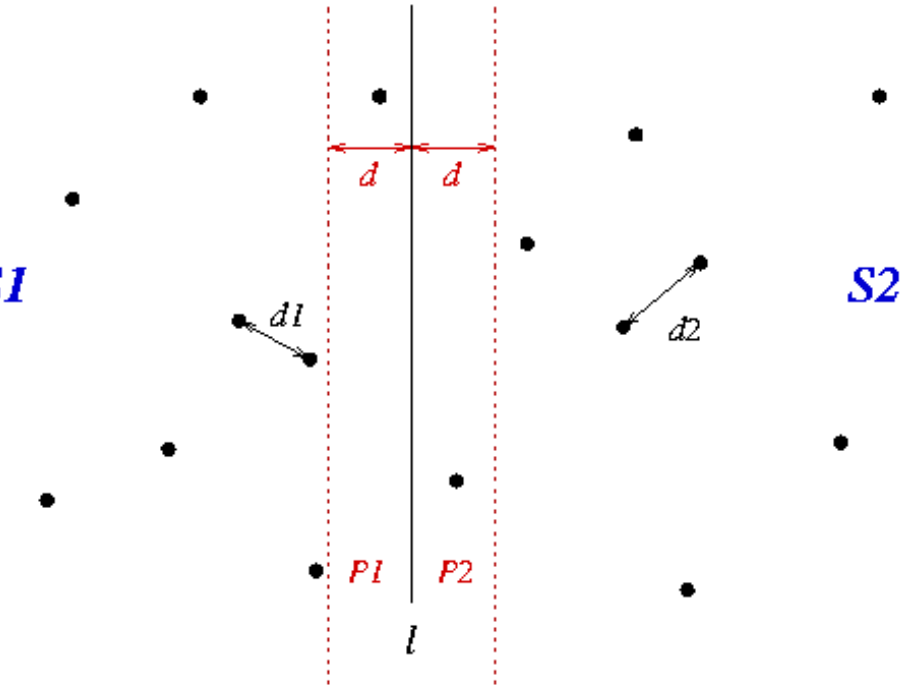

=> (din T. Master) $T(n) = \Theta(n * \log n)$

# Divide et Impera – alte exemple (I)

- Calculul puterii unui număr: $x^n$
  - Algoritm "clasic":
    - **Pentru** *i* **de la** 1 **la** *n* rez = rez * x;
    - **Întoarce** rez.
  - Complexitate: $\Theta(n)$
  - Algoritm Divide et Impera:
    - **Dacă** n este par
      - **Atunci Întoarce** $x^{n/2} * x^{n/2}$
      - **Altfel** (n este impar) **Întoarce** $x * x^{(n-1)/2} * x^{(n-1)/2}$
  - Complexitate: $T(n) = T(n/2) + \Theta(1) = \Theta(\log n)$

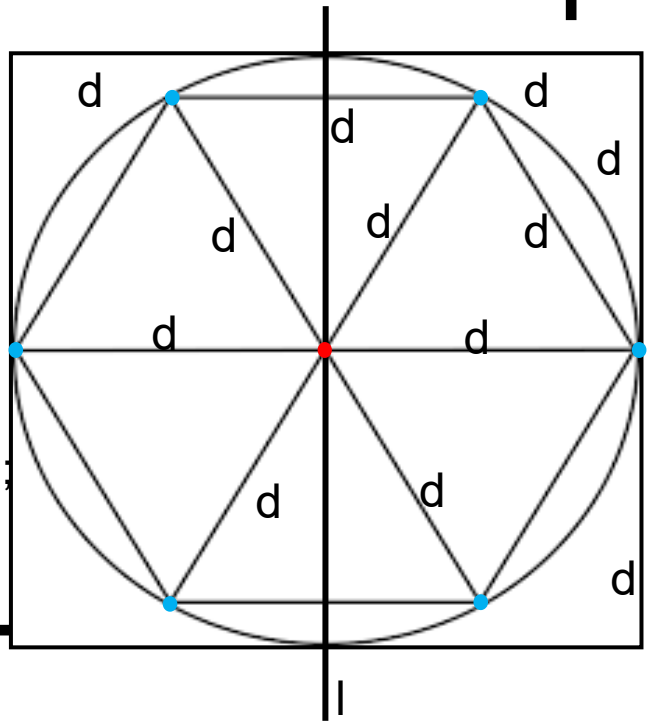
# Divide et Impera – alte exemple (II)

- Calculul celei mai scurte distanțe între 2 puncte de pe o dreaptă.
- Calculul celei mai scurte distanțe între 2 puncte din plan (http://www.cs.ucsb.edu/~suri/cs235/ClosestPair.pdf)
- Algoritmul naiv – $\Theta(n^2)$



# Divide et Impera – alte exemple (III)

- Sortează punctele în ordinea crescătoare a coordonatei x ($\Theta(n \log n)$);
- Împărțim setul de puncte în 2 seturi de dimensiune egală și calculăm recursiv distanța minimă în fiecare set (l = linia ce împarte cele 2 seturi, d = distanța minimă calculată în cele 2 seturi);
- Elimină punctele care sunt plasate la distanță de $l > d$;
- Sortează punctele rămase după coordonata y;
- Calculează distanțele de la fiecare punct rămas la cei 6 vecini (nu pot fi mai mulți) cu coordonata în intervalul [p–d, p+d], unde p e coordonata punctului;
- Dacă găsește o distantă < d, atunci actualizează d.



# Divide et Impera – Temă de gândire (1)

- Se dă o mulțime M de numere întregi și un număr x. Se cere să se determine dacă există a, b $\in$ M a.î. a + b = x.
- Algoritmul propus trebuie să aibă complexitatea θ(nlogn).
- Temele de la curs sunt facultative!☺



# Divide et Impera – Temă de gândire (2)

- Cum sortăm un vector fără interclasare sau alte operații complexe în etapele de împărțire a problemei și de combinare a soluțiilor?

- Hint: se folosesc trei apeluri recursive!
- Observație! Algoritmul este ineficient, dar este interesant!



# Divide et Impera – Temă de gândire (3)

- Cum se găseste eficient elementul median al unui vector?
- Problema mai generală: cum se găsește al k-lea cel mai mic (sau cel mai mare) element dintr-un vector?
- Eficient înseamnă $\Theta(n)$
- http://www.cs.rit.edu/~ib/Classes/CS515_Spring12-13/Slides/022-SelectMasterThm.pdf



# ÎNTREBĂRI?

